

IQ/MAX приложения и логика работы для проектирования

Executive summary

IQ/MAX по смыслу — это не набор разрозненных экранов, а единое **рабочее пространство дилингового пульта**: быстрые действия, справочники, мониторинг живой активности, управление активным звонком и отдельный слой многоканального прослушивания/ talkback. Сам продуктовый каркас типичен для trading turret: много линий, много одновременных аудиоканалов, несколько устройств ввода/вывода и постоянный визуальный контроль текущей связи. ¹

Для проектирования удобно думать так:

- **Snapshots / Navbar** — собирают рабочее место.
- **Favorites** — дают быстрые кнопки действий.
- **Directory** — помогает найти человека и выбрать способ связи.
- **Call History** — работает с прошлыми событиями звонков.
- **Activity Monitor** — показывает важное и текущее в live-режиме.
- **Footer / Handset Overlay** — всегда управляет активным звонком и устройствами.
- **Speakers** — отдельная модель мониторинга линий/ресурсов и talkback.
- **Settings** — настраивает все правила поведения.

В вашем текущем приложении уже есть близкие аналоги почти всех этих зон: workspace-каркас, contacts/phonebook/history, call queue, pinned calls, группы pinned calls, media devices, line keys и settings. Но workspace пока работает на моках, footer остается каркасом, а сильнее всего реализованы телефония, pinned calls, группы и устройства. turn0file0

Каркас приложения и главные сущности

Как читать IQ/MAX по уровням

Самая полезная модель такая:

Workspace / Snapshot

→ какие приложения открыты и как разложены

App

→ Favorites / Directory / History / Activity Monitor / Speakers / Settings

Domain entities

→ Contact / PointOfContact / LineResource / CallSession / SpeakerChannel / Device

Media control

→ AudioRoute / active device / mute / PTT / PTL

Главная ошибка — смешивать **контакт**, **линию**, **звонок**, **speaker channel** и **устройство**. Это разные сущности.

Ключевые сущности

```
type Contact = {
  id: string
  name: string
  presence?: 'available' | 'busy' | 'away' | 'dnd' | 'unavailable'
  pocs: PointOfContact[]
}

type PointOfContact = {
  id: string
  kind: 'intercom' | 'privateLine' | 'speedDial' | 'mobile' | 'sip' |
  'conference'
  value: string
  preferred?: boolean
}

type LineResource = {
  id: string
  kind: 'privateLine' | 'sharedLine' | 'dialToneLine' | 'openConnexion' |
  'hoot'
  access: 'none' | 'listen' | 'talk'
}

type CallSession = {
  id: string
  type: 'lineCall' | 'intercom' | 'conference' | 'speakerTalkback'
  state: 'dialing' | 'ringing' | 'active' | 'held' | 'ended'
  contactId?: string
  lineId?: string
}

type FavoriteButton = {
  id: string
  type: 'line' | 'contact' | 'poc' | 'conference' | 'function' | 'diversion'
  targetId?: string
  action: 'call' | 'answer' | 'open' | 'toggle' | 'redial'
}

type SpeakerChannel = {
  id: string
  assignedLineId?: string
  state: 'off' | 'on' | 'ptt' | 'ptl' | 'muted' | 'disabled'
  mediaMode:
```

```

    | 'txRxHfm'
    | 'txLeftRxHfm'
    | 'txRightRxHfm'
    | 'txRxLeftWhenPtt'
    | 'txRxRightWhenPtt'
}

type SpeakerGroup = {
  id: string
  name: string
  channelIds: string[]
  latchType?: 'ptt' | 'ptl' | 'slide'
}

type Device = {
  id: string
  kind: 'leftHandset' | 'rightHandset' | 'gooseneckMic' | 'hfmSpeaker' |
  'externalSpeaker'
  state: 'ready' | 'busy' | 'missing'
}

type AudioRoute = {
  id: string
  ownerType: 'callSession' | 'speakerChannel' | 'speakerGroup'
  ownerId: string
  input?: Device['kind']
  output: Device['kind'][]
  mutedInput?: boolean
  mutedOutput?: boolean
}

```

Сводка по приложениям

App	Что делает	С чем связано	Главные ограничения
Snapshots	сохраняет раскладку приложений	все apps	не управляет звонком напрямую
Favorites	быстрые кнопки действий	POC, line, conference, function	не заменяет Directory и Footer
Directory	поиск контактов и выбор канала связи	Contact, POC, presence	это справочник, а не live-monitor
Call History	прошлые call events	CallSession events, recordings	не live, а история
Activity Monitor	важные live-события	lines, calls, priority	показывает подмножество, не весь каталог

App	Что делает	С чем связано	Главные ограничения
Footer / Handset Overlay	активный звонок и устройства	active session, handset, HFM, volume	foreground-control, не поиск
Speakers	мониторинг линий и talkback	speaker channels, groups, devices	работает вокруг ресурсов/линий, не вокруг людей
Settings	конфиг поведения	preferences, devices, routing, line keys	многое зависит от backend/controller

Операционные приложения

Favorites

Что делает.

Это главная панель быстрых кнопок. Не “адресная книга”, а **сетка часто используемых действий**.

С чем связано.

Favorites опирается на:

- PointOfContact контакта;
- LineResource ;
- conference entry;
- function button;
- diversion button.

Что обычно может быть кнопкой.

Тип	Смысл
Line	занять / ответить / наблюдать линию
Contact / POC	быстрый вызов по выбранному способу связи
Conference	быстро открыть или набрать конференцию
Function	redial, hold, release, mute и т. п.
Diversion	быстрые сценарии переадресации

Какие ограничения.

- Favorites не нужен для глубокого поиска — этим занимается Directory.
- Favorites не является главным местом управления уже активным звонком — этим занимается Footer.
- Лимит по числу кнопок/листов зависит от конкретной конфигурации; в доступном сейчас источнике это **не перепроверено**.

Какие сущности участвуют.

FavoriteButton
PointOfContact
LineResource
Conference
FunctionAction

Пример сценария.

Пользователь открывает Favorites
→ нажимает кнопку "Иванов"
→ система берет preferred POC
→ стартует CallSession
→ активный звонок уходит в Footer / Handset Overlay

Связь с вашим продуктом.

У вас отдельной полноценной Favorites-панели пока нет, но есть основа для нее: контакты, phonebook, line keys, pinned calls, call queue и hardware bindings. Это позволяет строить Favorites как слой быстрых кнопок поверх уже существующих сущностей. [fileciteurlturn0file0](#)

Directory

Что делает.

Directory — это **справочник контактов**, а не кнопочная панель. Его роль: найти человека, увидеть его доступность и выбрать способ связи.

С чем связано.

- Contact
- PointOfContact
- Presence
- PersonalContact
- Favorites (добавление POC в быстрые кнопки)

Что пользователь делает в Directory.

- ищет контакт;
- смотрит presence;
- открывает карточку;
- выбирает нужный POC;
- звонит по preferred POC;
- добавляет POC в Favorites;
- работает с личными контактами.

Какие ограничения.

- Enterprise directory обычно ближе к read-only.
- Directory не управляет активным звонком.
- Presence — это статус человека, а не статус линии.

Какие сущности участвуют.

Contact
PointOfContact
Presence
DirectoryGroup
PersonalContact

Пример сценария.

Пользователь ищет "Петров"
→ видит presence = Busy
→ открывает карточку
→ выбирает Intercom вместо мобильного
→ создает вызов

Связь с вашим продуктом.

У вас эту роль уже частично выполняют Contacts + Phonebook: карточки контактов, телефонная книга, вкладки, подписки на статусы контактов и линий, frequently called и серверные группы. Это очень близкий аналог будущего Directory. turn0file0

Call History

Что делает.

Call History — это журнал **прошлых** событий общения.

С чем связано.

- CallHistoryEvent
- CallSession
- Contact
- Recording
- Favorites / Directory

Что пользователь делает.

- смотрит missed / incoming / outgoing;
- фильтрует и ищет;
- открывает детали;
- перезванивает;
- создает контакт;
- добавляет номер в Favorites.

Какие ограничения.

- Это не live-монитор.
- Это не control panel активного вызова.
- Глубина хранения истории зависит от backend и retention policy; здесь это **не уточнено**.

Какие сущности участвуют.

```
CallHistoryEvent  
ContactRef  
RecordingRef  
CallResult
```

Пример сценария.

```
Пользователь открывает History  
→ видит пропущенный вызов от 1001  
→ нажимает Redial  
→ создается новый CallSession
```

Связь с вашим продуктом.

У вас история уже есть как отдельный раздел с IM-based загрузкой, live-обновлением, таблицей, пагинацией и данными о длительности/записи. Это хороший прямой аналог.

filecite turn0file0

Activity Monitor

Что делает.

Activity Monitor — это не просто еще одна сетка кнопок. Это экран **важного live-состояния**.

Обычно он делится на:

- **Fixed items** — пользователь сам закрепил важные объекты;
- **Floating items** — система сама показывает активные и приоритетные события.

С чем связано.

- FavoriteButton
- LineResource
- CallSession
- Priority
- Incoming/Held/Off-screen activity

Что пользователь делает.

- быстро замечает incoming/held/priority activity;
- отвечает на вызов;
- возвращает hold;
- следит за важными monitored resources.

Какие ограничения.

- Это не каталог всего.
- Это не заменяет Favorites.
- Это не редактирование контактов/кнопок; это live-observer.

Какие сущности участвуют.

```
MonitorItem  
MonitorSource  
PriorityLevel  
CallState
```

Пример сценария.

```
Пользователь в другом app  
→ на важной линии возникает incoming call  
→ в Activity Monitor появляется floating item  
→ пользователь кликает по нему  
→ звонок становится foreground session
```

Связь с вашим продуктом.

Прямого аналога пока нет, но его логично собирать из уже реализованных у вас call queue, pinned calls и статусов активных SIP-сессий. Очередь вызовов у вас уже показывает активные неприкрепленные сессии и открывает call manager по типу состояния. [filecite](#)[turn0file0](#)

Контроль звонка и аудио

Footer и Handset Overlay

Что делает.

Это главный **глобальный control layer** активного звонка. Он должен быть доступен всегда.

С чем связано.

- `ActiveSessionContext`
- `DeviceSlot` (left / right / HFM)
- `Volume`
- `Mute`
- `UserPresence`
- `CallNotification`

Что пользователь делает.

- видит текущую активную сессию;
- управляет hold / release / mute;
- понимает, какой handset/HFM сейчас используется;
- получает call notifications;
- видит свой user badge / presence.

Какие ограничения.

- Footer работает вокруг **foreground context**.
- Он не предназначен для поиска людей.

- Он не должен дублировать функциональность Directory или Speakers.

Какие сущности участвуют.

```
ActiveSessionContext
HandsetSlot
HfmState
VolumeState
UserBadge
```

Пример сценария.

```
Пользователь уже в разговоре
→ новый звонок приходит на другую линию
→ footer показывает incoming notification
→ пользователь берёт звонок или ставит текущий на hold
```

Связь с вашим продуктом.

У вас footer уже встроен в shell и присутствует на внутренних экранах, но его бизнес-роль как “центрального call control layer” еще не доведена до уровня IQ/MAX. В документе прямо видно, что footer пока скорее каркас. `filecite` `turnOfffile0`

Speakers

Что делает.

Speakers — это не список людей. Это приложение для **мониторинга аудиоресурсов** и быстрого talkback.

Самая важная формула:

```
Speaker ON = я слышу
PTT / PTL = меня слышат
```

Что можно добавить в Speakers

В модель IQ/MAX туда по-хорошему добавляют **не абонента**, а **линию/ресурс**:

- private line;
- shared/dial line;
- dial tone line;
- open connexion / hoot;
- другой разрешенный audio resource.

То есть правильно мыслить так:

```
Speaker Channel 1 → LineResource 1111
```

а не так:

Speaker Channel 1 → Иванов, Петров, Сидоров

Если у линии сейчас multiparty-разговор, speaker channel просто слышит весь аудиопоток этой линии.

Канальная модель

Один `SpeakerChannel` = один назначенный ресурс.

Типовая модель состояния:

off	— канал выключен
on	— канал слушается
ptt	— временная передача
ptl	— защелкнутая передача
muted	— звук канала выключен локально
disabled	— канал недоступен из-за прав или device conflict

Media device modes

Режим	Что означает
<code>txRxHfm</code>	говорить через HFM/gooseneck, слушать через hands-free speaker
<code>txLeftRxHfm</code>	говорить через левую трубку, слушать через speaker
<code>txRightRxHfm</code>	говорить через правую трубку, слушать через speaker
<code>txRxLeftWhenPtt</code>	при PTT/PTL и говорить, и слушать через левую трубку
<code>txRxRightWhenPtt</code>	при PTT/PTL и говорить, и слушать через правую трубку

Что такое PTT и PTL

- **PTT** — Push-to-Talk: говорю, пока удерживаю.
- **PTL** — Push-to-Latch: говорю, пока latch не снят.
- **Group talkback** — говорю сразу в группу speaker channels.

Groups

`SpeakerGroup` — это группа **каналов**, а не людей.

Пример:

Group "Desk A"
- Channel 1 / Line 1111

- Channel 2 / Line 2222
- Channel 3 / Hoot 5

Что дает группа:

- общий talkback;
- общий latch mode;
- групповое оперативное управление.

Точный лимит количества групп в доступном сейчас источнике **не перепроверен**.

Volume и mute rules

Главные правила:

1. Output можно миксовать.

Пользователь может слушать несколько speaker channels одновременно.

2. Input нельзя открывать хаотично.

Микрофон должен быть у одного активного talk route либо у явно выбранной группы.

3. Mute route-scoped.

Mute относится не ко всему физическому миру сразу, а к конкретному маршруту.

4. Speaker ON не равен microphone ON.

Типовые mute-состояния канала:

Состояние	Что значит
local muted	я сам выключил звук канала у себя
muted by my handset	эта же линия у меня занята foreground-call через handset
muted by other handset	другой пользователь занял линию через handset
muted by other microphone	другой talk-route сейчас обладает приоритетом

Отдельная регулировка громкости группы в виде полноценного master-slider для группы в доступном сейчас источнике **не подтверждена**; по рабочей модели безопаснее считать, что базовая громкость — на уровне канала, а группа дает прежде всего логическое групповое mute/talkback.

Конфликты устройств

Это ключевой блок логики.

Основные правила:

- один handset не может быть полноценным foreground Tx/Rx сразу для двух независимых активных сценариев;

- handset call имеет приоритет над speaker monitoring;
- если устройство занято, PTT/PTL на конфликтующем канале должен быть disabled, отклонен или сопровождаться blocking feedback;
- HFM/gooseneck — это общее устройство, которое может временно переключаться между hands-free call и speaker talkback.

HFM / gooseneck rules

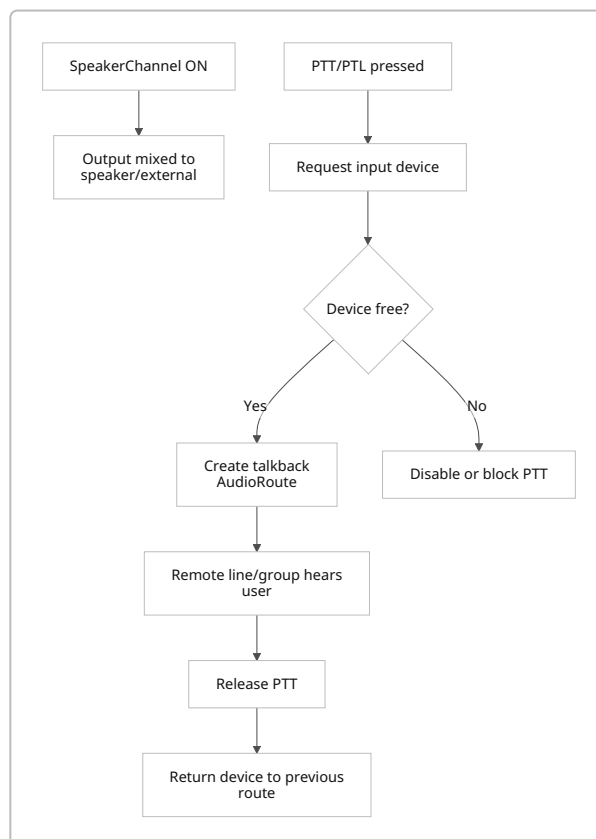
Gooseneck/HFM нужен не только для Speakers. Он участвует как минимум в трех режимах:

- hands-free call;
- hands-free intercom;
- speaker talkback.

Правильная модель при конфликте:

HFM call active
 + user presses PTT on Speaker Channel
 → mic временно уходит в speaker talkback
 → основной call остается активным
 → после отпускания PTT mic возвращается обратно

Правила AudioRoute для Speakers



Пример сценария

Пользователь включил 5 speaker channels
→ слышит 5 линий одновременно
→ нажимает PTT на Channel 3
→ только Channel 3 получает его голос
→ отпускает PTT
→ снова остается только listening

Связь с вашим продуктом.

Ваши pinned calls и pinned groups — очень близкий функциональный аналог Speakers, но с другой доменной моделью. У вас уже есть до 16 pinned slots, per-slot mic/volume, pinned device, синхронизация сессий, группы из pinned slots, групповое управление mic/volume и хранение всего этого в preferences. Это сильная база. Главное отличие: у вас сейчас завешивается чаще pServed /контакт/конференция, а в IQ/MAX Speakers обычно построены вокруг line/resource model. turn0file0

Практический вывод для проектирования

Если переносить логику в ваш продукт, лучше думать так:

Pinned Call Slot → Pinned Channel
Pinned Group → Speaker Group
Pinned Device → media device for channel/group

И постепенно переходить от модели “завесили абонента” к модели “канал слушает аудиоресурс”.

Конфигурационные приложения

Snapshots

Что делает.

Snapshots сохраняют **рабочие раскладки приложений**.

Это слой не о звонках, а о рабочем месте:

- какие apps открыты;
- где они стоят;
- какая раскладка нужна для конкретной задачи.

С чем связано.

- Workspace
- Snapshot
- AppInstance
- WidgetPlacement

Какие ограничения.

- Snapshot не содержит сам звонок как объект контроля.
- Он только восстанавливает интерфейсный контекст.
- Точный лимит snapshot'ов зависит от конкретной системы и здесь **не перепроверен**.

Пример сценария.

Пользователь переключается на snapshot "Open Market"
→ открываются Favorites + Activity Monitor + Speakers
→ вечером переключается на "Support"
→ layout меняется, но бизнес-данные приложений те же

Связь с вашим продуктом.

У вас уже есть workspace route, layout types, widget registry и 3 моковых workspace, но пока нет backend-driven CRUD и полноценного сохранения рабочих столов. Это прямой задел под Snapshots. fileciteturn0file0

Settings

Что делает.

Settings задает пользовательские и системные правила приложения.

С чем связано.

- UserPreferences
- MediaDeviceProfile
- ForwardingRule
- PriorityRule
- ContactTab
- LineKeyBinding

Что обычно настраивается.

- пароль / account info;
- incoming call processing / diversion;
- call handling;
- ringtone / volume;
- media devices;
- line keys;
- contact tabs;
- prioritization;
- diagnostics / system info.

Какие ограничения.

- часть настроек зависит от backend;
- часть — от аппаратного controller;
- часть — чисто локальные preference.

Пример сценария.

Пользователь открывает Settings
→ меняет media device profile
→ биндинг line keys
→ настройки рингтона и incoming processing
→ новые правила начинают использовать Footer / Calls / Speakers

Связь с вашим продуктом.

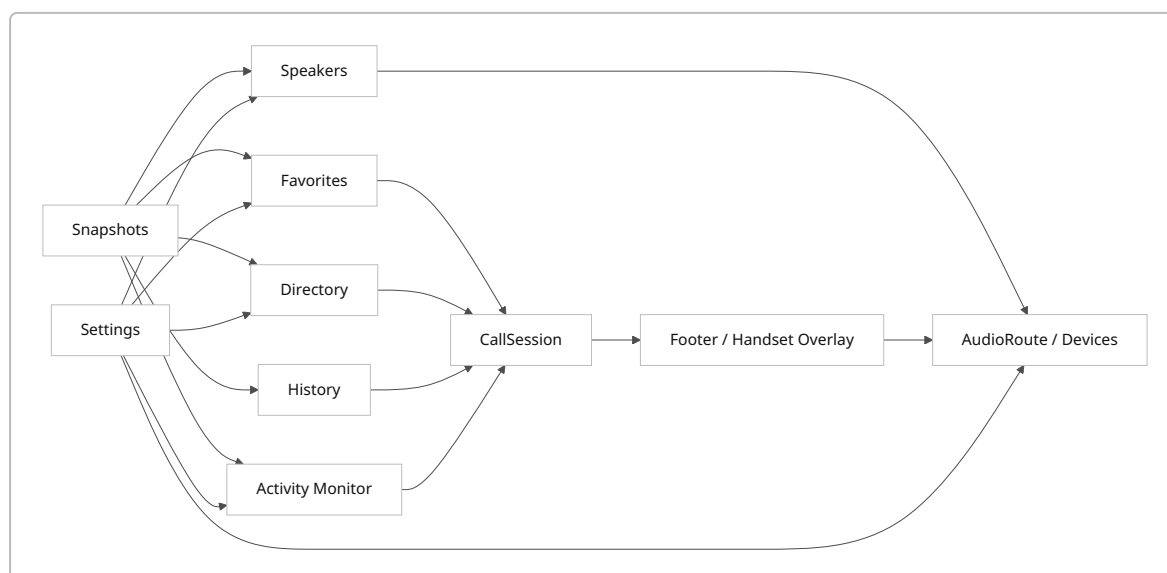
Этот слой у вас уже довольно зрелый: main settings, media devices, incoming call processing, contact tabs, prioritization, line keys и сохранение prefs уже реализованы; black/white list пока заглушка.

Особенно важно, что у вас уже есть связь settings ↔ controller ↔ devices ↔ line keys.

filecite\turn0file0

Связи между приложениями и привязка к вашему продукту

Как приложения связаны между собой



Что в вашем продукте уже есть и куда это ставить

IQ/MAX app	Ваш ближайший аналог	Состояние
Snapshots	workspace	каркас есть, backend model нет filecite\turn0file0
Favorites	будущая quick-call panel + line keys	как отдельный полноценный app еще не оформлено filecite\turn0file0
Directory	contacts + phonebook + tabs	хорошая база уже есть filecite\turn0file0

IQ/MAX app	Ваш ближайший аналог	Состояние
Call History	history	уже фактически реализовано filecite turn0file0
Activity Monitor	call queue + pinned-aware live monitor	нужно собрать как отдельную бизнес-фичу filecite turn0file0
Footer / Handset Overlay	footer	пока каркас, нужно усиление filecite turn0file0
Speakers	pinned calls + groups + pinned device	самая сильная база, но доменная модель пока чуть другая filecite turn0file0
Settings	settings	уже почти полноценный слой filecite turn0file0

Главное итоговое правило

Если смотреть на IQ/MAX “по порядку”, логика приложения такая:

1. **Snapshots** собирают рабочее место.
2. **Favorites** дают быстрые действия.
3. **Directory** и **History** дают выбор, кому и как позвонить.
4. **Activity Monitor** показывает, что важно прямо сейчас.
5. **Footer / Handset Overlay** управляет текущим foreground-call.
6. **Speakers** управляют многоканальным listening/talkback.
7. **Settings** определяют правила устройств, звонков, routing и поведения интерфейса.

Для вашего проекта это означает очень конкретную целевую модель:

- **Contacts/Phonebook** развиваются в **Directory**.
- **Quick call panel** развивается в **Favorites**.
- **Call queue + live status** развиваются в **Activity Monitor**.
- **Pinned calls + groups** развиваются в **Speakers**.
- **Footer** становится главным call-control hub.
- **Workspace** становится **Snapshots**.

Именно в таком порядке архитектура становится логичной и не расползается по страницам.

¹ https://en.wikipedia.org/wiki/IPC_Systems
https://en.wikipedia.org/wiki/IPC_Systems